



**Universitat
de Lleida**



ESCOLA
POLITÈCNICA SUPERIOR
UNIVERSITAT DE LLEIDA



Orchestration Trade-offs in Edge Computing

Informe de Seguimiento: Evaluación de Protocolos
de Red y Pipeline MLOps

XiaoLong Ji

01/07/2026

Systemd vs kubernetes

1. Ansible + Systemd + Podman

Comparativa protocolos de comunicaciones en Systemd + podman y obtener las metricas y números.

- Infraestructura (Ansible)
- Entrenamiento de IA en nodo Master
- Protocolos de comunicación + serialización
- Métricas y resultados

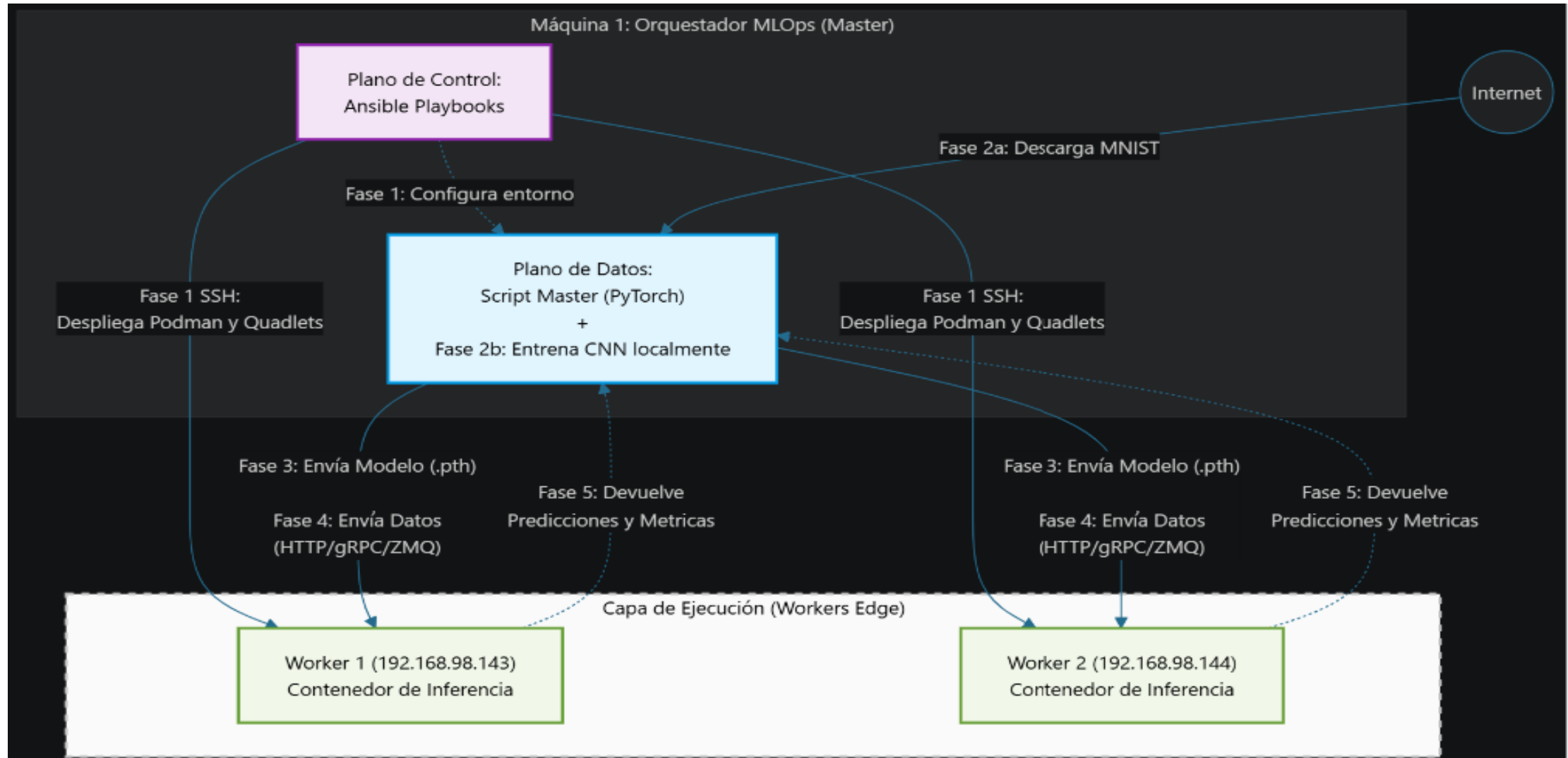
Tarea

Amb el dataset MNIST

(<https://www.tensorflow.org/datasets/catalog/mnist>), has de fer:

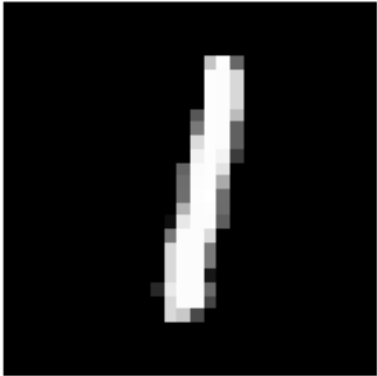
1. Amb aquest dataset s'ha de entrenar un model IA amb PyTorch, que s'enviaran als N nodes fills
2. Els nodes fills faran una tasca sobre aquest dataset, que és reconèixer el contingut de la imatge amb el model IA
3. Els nodes fills han de retornar al pare el resultat de la tasca. En aquest cas el contingut de la imatge

Diagrama

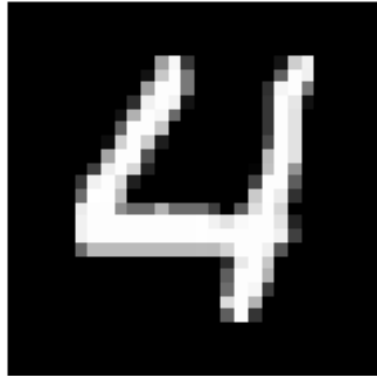


Dataset MNIST

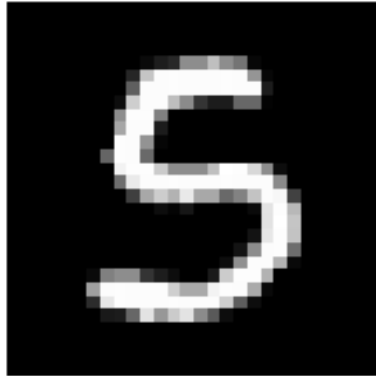
Label: 1



Label: 4



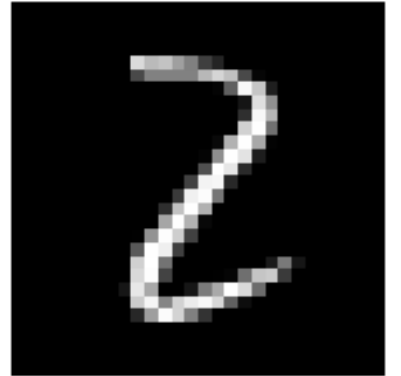
Label: 5



Label: 9



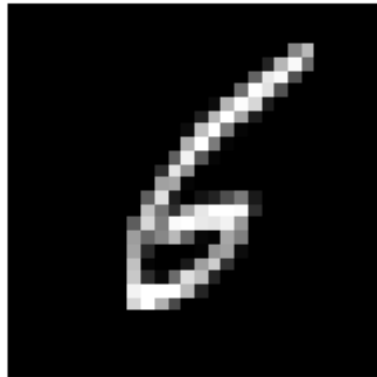
Label: 2



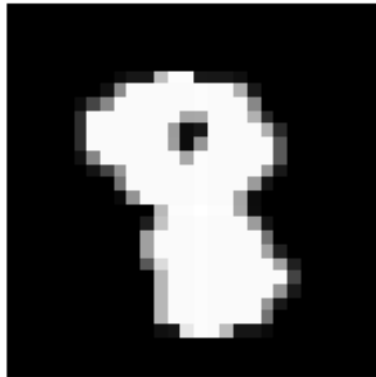
Label: 2



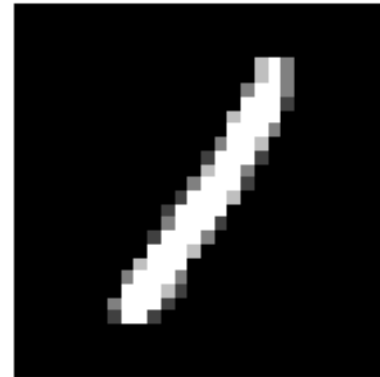
Label: 6



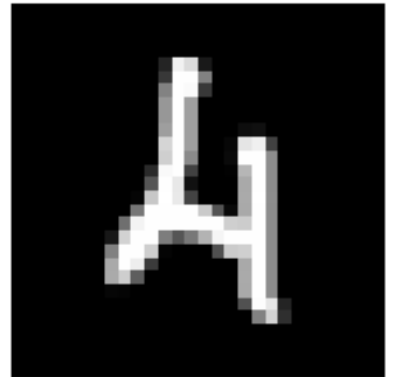
Label: 8



Label: 1



Label: 4



PyTorch – Modelo de redes neuronales

1. CNN over MLP, ya que mantiene los píxeles de las imágenes sin manipularlo

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(2, 2)
        # Añadimos Dropout aquí
        self.dropout = nn.Dropout(0.25)

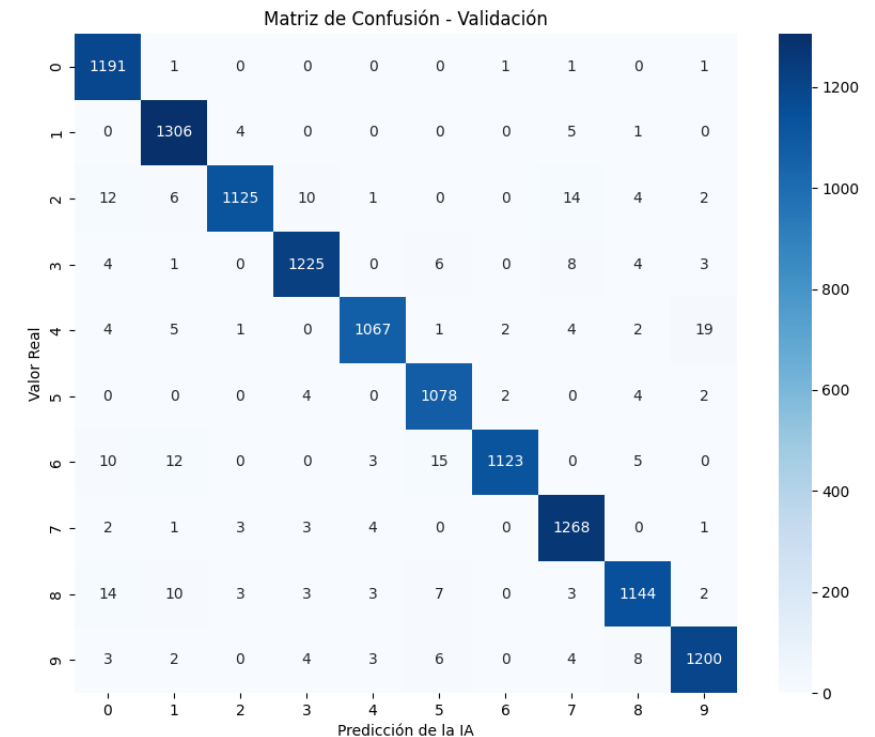
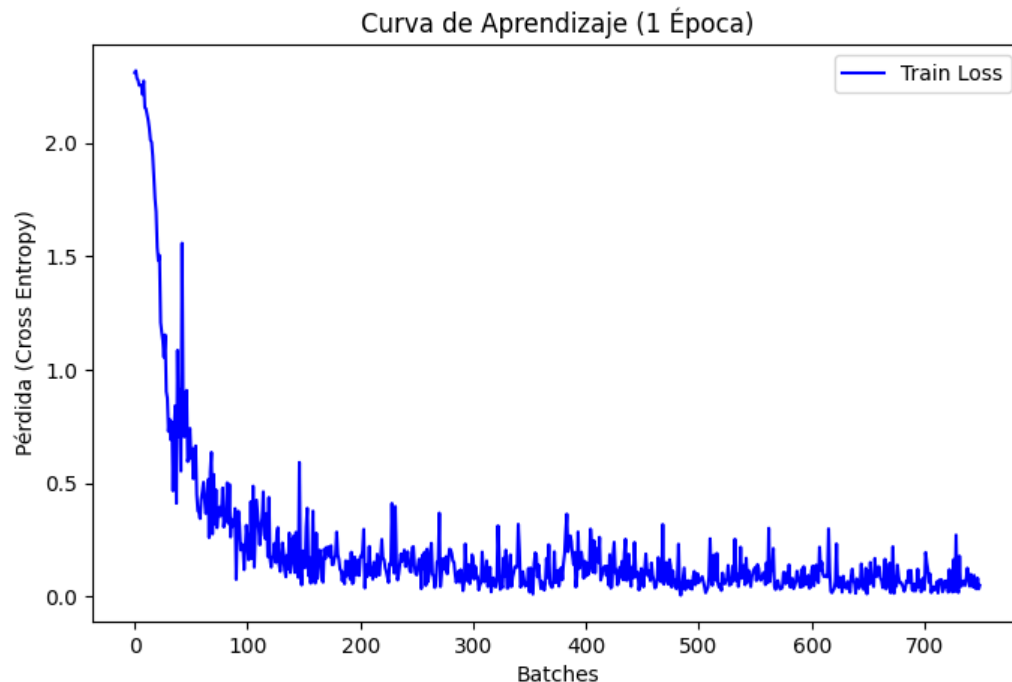
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)
```

```
def forward(self, x):
    x = self.pool(self.relu(self.conv1(x)))
    x = self.dropout(x) # Aplicamos dropout
    x = self.pool(self.relu(self.conv2(x)))
    x = x.view(-1, 64 * 7 * 7)
    x = self.relu(self.fc1(x))
    x = self.fc2(x)
    return x
```

PyTorch – Fase entrenamiento

2. El dataset MNIST cuenta con **60.000 imágenes** para entrenamiento:

- Conjunto de **Entrenamiento (80%): 48.000** imágenes.
- Conjunto de **Validación (20%): 12.000** imágenes.



PyTorch – Fase evaluación del modelo

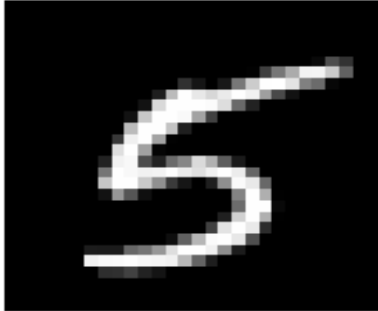
3. --- MÉTRICAS DEL MODELO (IA) ---

Precisión (Validation Accuracy): 97.36% (La precisión ronda entre 97% - 98%)

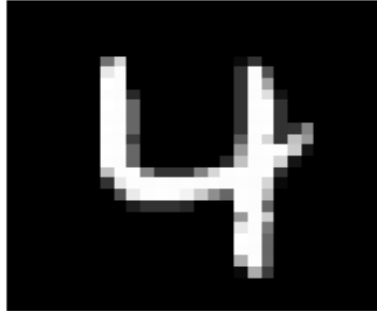
Tiempo de entrenamiento: 7.69 s

Peso del artefacto (.pth): 1.61 MB

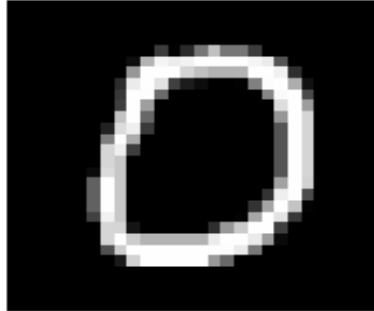
Pred: 5, True: 5



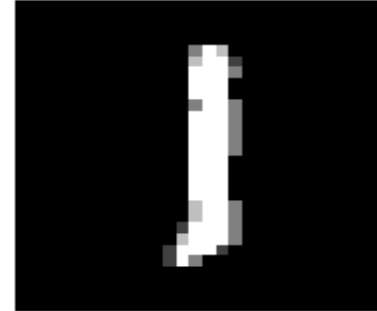
Pred: 4, True: 4



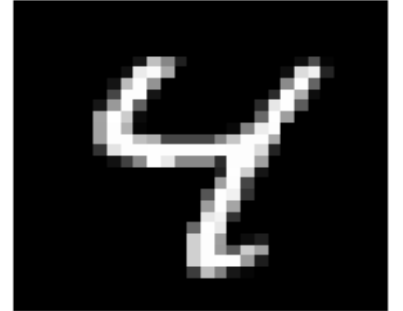
Pred: 0, True: 0



Pred: 1, True: 1



Pred: 4, True: 4



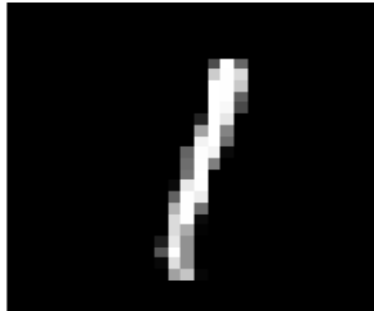
Pred: 3, True: 3



Pred: 3, True: 3



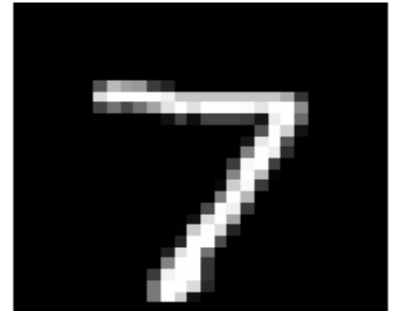
Pred: 1, True: 1



Pred: 2, True: 2



Pred: 7, True: 7



Resultados: Comparativa Infraestructura

--- MÉTRICAS DE INFRAESTRUCTURA http json---

[OK] T_deploy total: 20.24 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 6.34% | RAM Reposo: 32.16MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 824.08 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 3.69% | RAM Reposo: 32.17MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 825.93 ms

--- MÉTRICAS DE INFRAESTRUCTURA grpc protobuf---

[OK] T_deploy total: 35.18 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 5.75% | RAM Reposo: 23.85MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 922.53 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 3.86% | RAM Reposo: 23.85MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 917.23 ms

--- MÉTRICAS DE INFRAESTRUCTURA ZMQ protobuf---

[OK] T_deploy total: 26.66 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 4.47% | RAM Reposo: 17.6MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 825.26 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 3.43% | RAM Reposo: 17.6MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 877.54 ms

--- MÉTRICAS DE INFRAESTRUCTURA ZMQ MessagePack---

[OK] T_deploy total: 29.45 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 3.94% | RAM Reposo: 16.14MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 810.84 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 2.38% | RAM Reposo: 16.14MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 937.31 ms

VS

--- MÉTRICAS DE INFRAESTRUCTURA http json---

[OK] T_deploy total: 177.38 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 23.26% | RAM Reposo: 169MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 498.74 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 13.53% | RAM Reposo: 169MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 471.46 ms

--- MÉTRICAS DE INFRAESTRUCTURA grpc protobuf---

[OK] T_deploy total: 171.25 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 22.55% | RAM Reposo: 156.5MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 538.18 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 12.77% | RAM Reposo: 156.5MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 592.79 ms

--- MÉTRICAS DE INFRAESTRUCTURA ZMQ protobuf---

[OK] T_deploy total: 175.11 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 18.76% | RAM Reposo: 154.2MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 526.47 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 10.41% | RAM Reposo: 154.2MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 571.64 ms

--- MÉTRICAS DE INFRAESTRUCTURA ZMQ MessagePack---

[OK] T_deploy total: 192.61 segundos

[OK] Nodo 192.168.98.143 - CPU Reposo: 19.37% | RAM Reposo: 152.8MB / 4.056GB

[OK] Nodo 192.168.98.143 - Tiempo Real Arranque: 510.95 ms

[OK] Nodo 192.168.98.144 - CPU Reposo: 11.22% | RAM Reposo: 152.8MB / 4.056GB

[OK] Nodo 192.168.98.144 - Tiempo Real Arranque: 454.55 ms

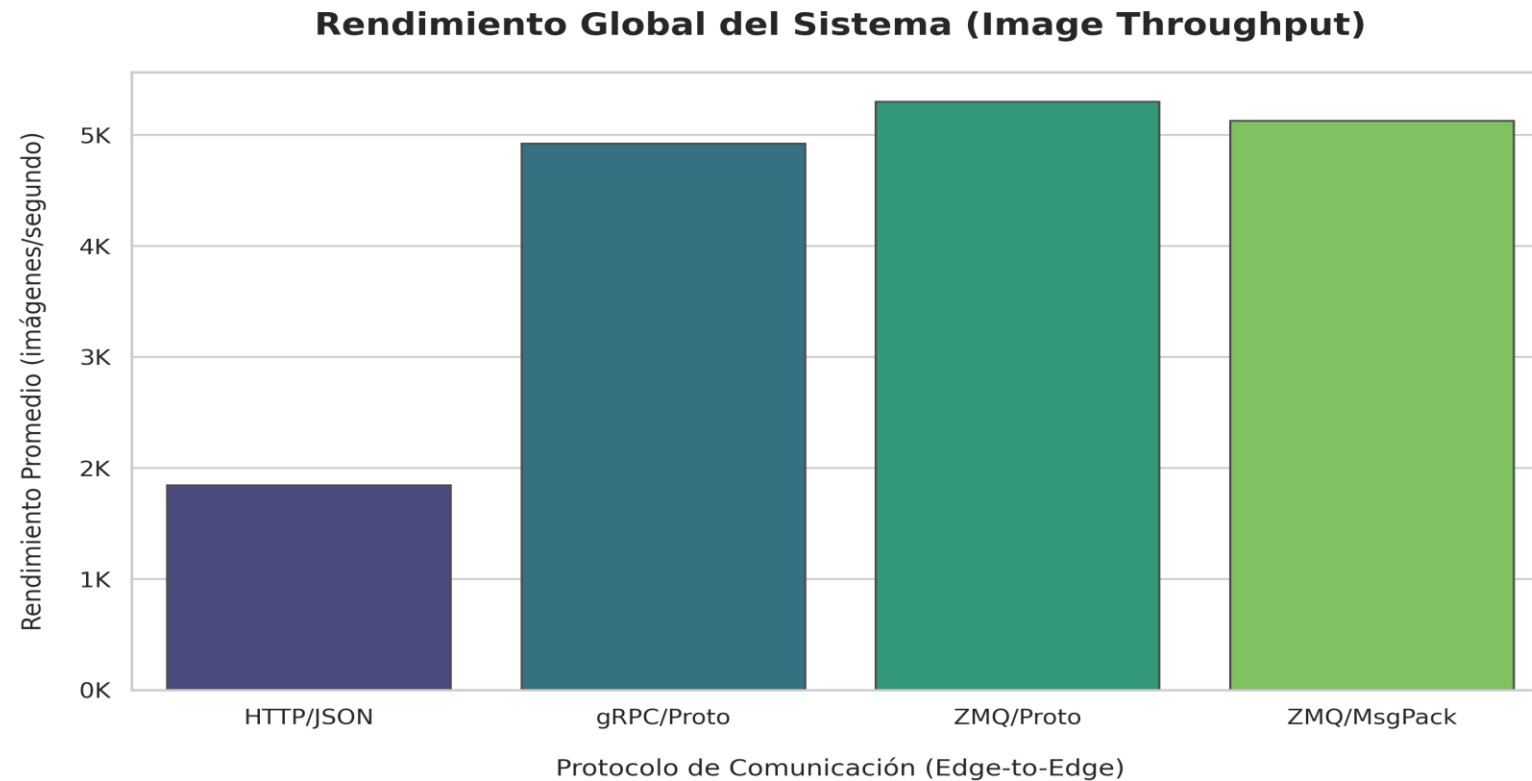
Incremento significativo en el tiempo de despliegue total (T_{deploy}), pasando de una media de **~28 segundos** en la tarea simple de conteo a **~180 segundos**

1. **Dependencias de Inferencia:** requieren la frameworks torch y sus dependencias de computación tensorial
2. **Carga de Artefactos:** El proceso de despliegue ahora incluye la distribución y carga en memoria de los pesos del modelo (best_model.pth)

Resultados: Comparativa protocolos - tabla

Métrica Consolidada	HTTP/JSON	gRPC (Protobuf)	ZeroMQ (Protobuf)	ZeroMQ (MessagePack)
Tiempo Total Promedio	1.13 s	0.48 s	0.46 s	0.48 s
Throughput Promedio	1845.01 img/s	4923.26 img/s	5300.36 img/s	5127.61 img/s
Latencia RTT Promedio	722.69 ms	467.50 ms	458.74 ms	470.81 ms
Tiempo T _{proc} Promedio	704.29 ms	433.12 ms	437.63 ms	449.49 ms
Pico Máximo RAM (Worker)	335.23 MB	311.71 MB	279.49 MB	269.69 MB
CPU Máxima (Worker)	152.57 %	193.96 %	194.88 %	194.91 %
Payload Total por Nodo	15.59 MB	2.99 MB	2.99 MB	2.99 MB

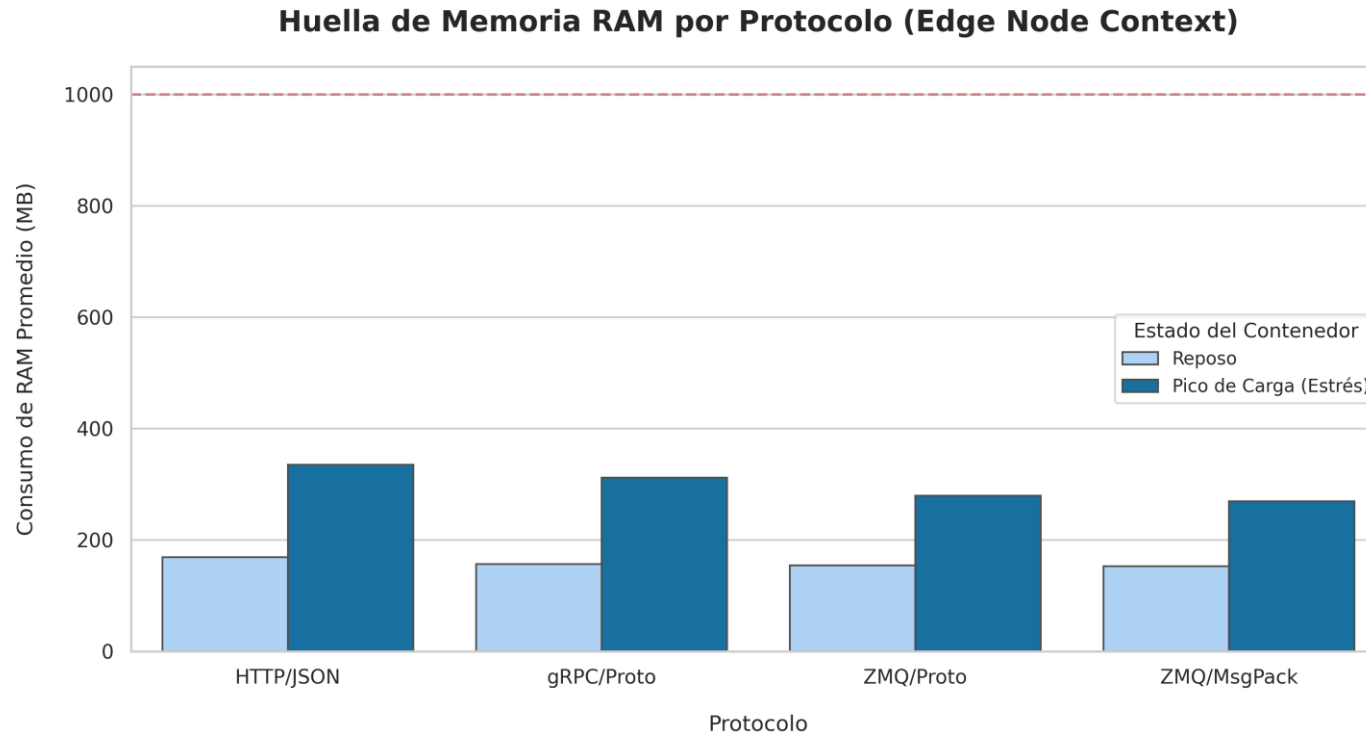
Resultados: Comparativa protocolos - Gráfica



Observación:

Se aprecia un salto exponencial en el throughput al pasar de protocolos basados en texto (JSON) a binarios (Protobuf/MsgPack), alcanzando el pico de eficiencia en la cuarta iteración.

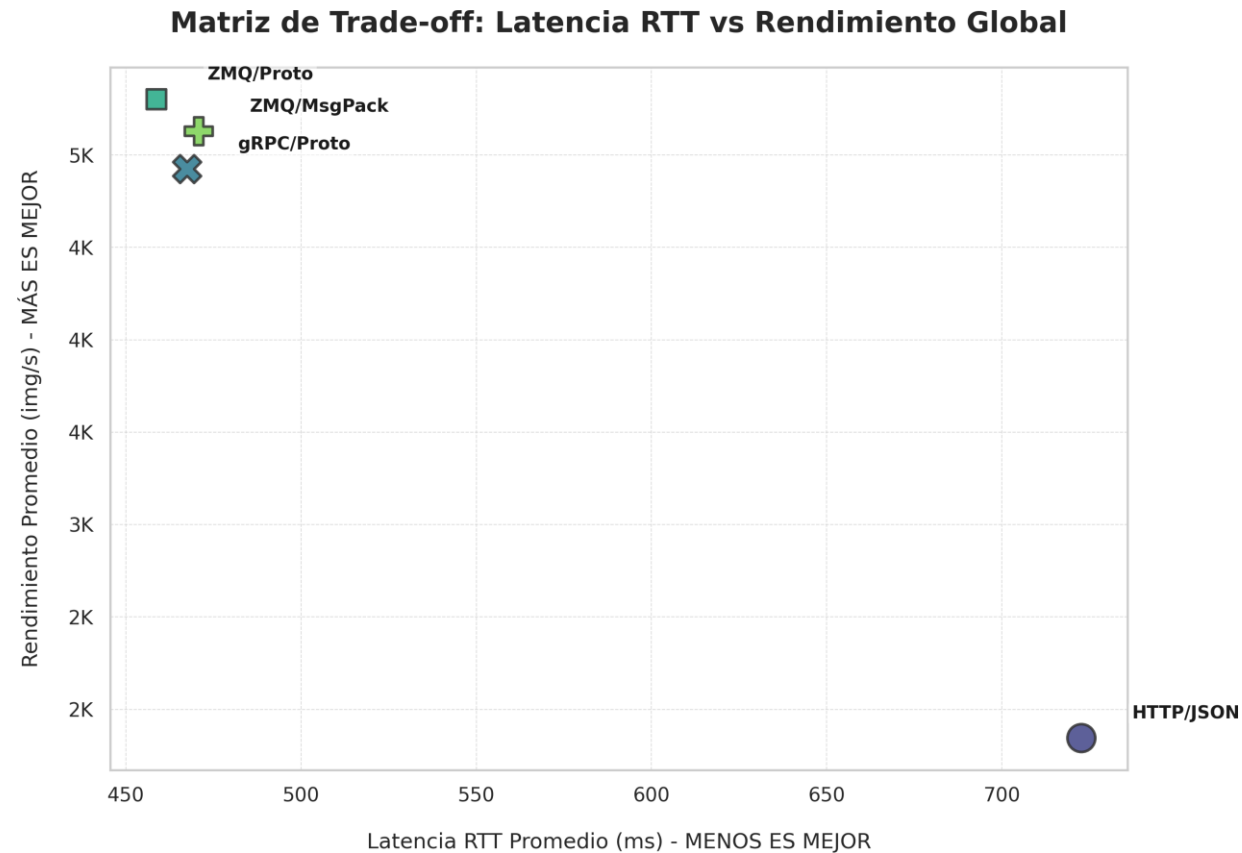
Resultados: Comparativa protocolos - Gráfica



Observación:

En esta fase, el ritmo lo marca la **capacidad de procesamiento (IA)** y no la red. La CNN realizar cálculos complejos para cada imagen. Esto introduce una espera en la CPU y hace que el consumo de RAM en reposo es constante

Resultados: Comparativa protocolos - Gráfica



Observación: El cuadrante superior izquierdo (Baja latencia, Alto throughput) es dominado por ZeroMQ + MessagePack, validando la arquitectura como la solución técnica óptima para el procesamiento distribuido en el Edge.

Conclusiones

- **CNN** porque no solo por precisión (97.98%), sino porque las CNN manejan tensores multidimensionales reales.
- **Una única época** fue suficiente ya que nuestra meta era validar la arquitectura de comunicaciones, no optimizar el modelo al milímetro.
- Aunque los binarios muestran resultados similares, optaría por **ZeroMQ + msgpack** ya que no hace falta establecer una estructura.